Kleisli Composition

Kleisli · Compose parseInt → enrichUser → authorize · Build with Waqar

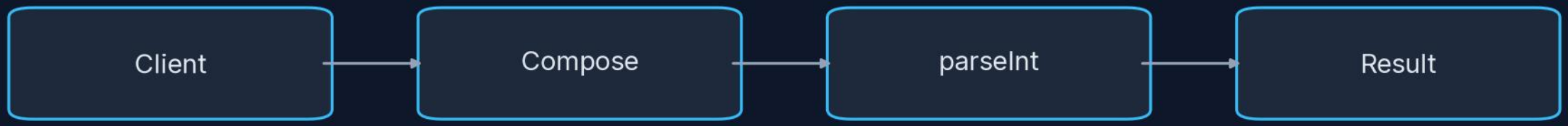
Today's Hook

- Problem: Compose parseInt → enrichUser → authorize
- Pattern: Kleisli — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Compose parseInt → enrichUser → authorize
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Kleisli: Compose parseInt → enrichUser → authorize

Real-World Scenario

- Running example: Compose parseInt → enrichUser → authorize
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.kleisli;

import com.buildwithwaqar.patterns.functional.kleisli.Kleisli;
import com.buildwithwaqar.patterns.functional.typeclass.OptionKind;
import dev.zazr.control.Option;
/** Problem: compose parseInt → *2 fallible pipeline. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Kleisli problem: fallible pipeline ===");
    Kleisli<OptionKind.Witness, String, Integer> parse = Kleisli.of(s -> {
      try { return OptionKind.lift(Option.some(Integer.parseInt(s))); }
      catch (Exception e) { return OptionKind.lift(Option.none()); }
    });
    Kleisli<OptionKind.Witness, Integer, Integer> twice = Kleisli.of(n -> OptionKind.lift(Option.some(n * 2)));
    var p = parse.andThen(OptionKind.MONAD, twice);
    System.out.println("ok=" + OptionKind.lower(p.apply("21")));
    System.out.println("bad=" + OptionKind.lower(p.apply("zz")));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.kleisli;

import com.buildwithwaqar.patterns.functional.typeclass.Kind;
import com.buildwithwaqar.patterns.functional.typeclass.Monad;

import java.util.function.Function;

/** Kleisli arrow A =&gt; F&lt;B&gt; — composable effectful functions. */
public record Kleisli<F, A, B>(Function<A, Kind<F, B>> run) {
    public Kind<F, B> apply(A a) { return run.apply(a); }

    public <C> Kleisli<F, A, C> andThen(Monad<F> M, Kleisli<F, B, C> next) {
        return new Kleisli<>(a -> M.flatMap(run.apply(a), next.run));
    }

    public static <F, A, B> Kleisli<F, A, B> of(Function<A, Kind<F, B>> f) {
        return new Kleisli<>(f);
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.kleisli;

import com.buildwithwaqar.patterns.functional.kleisli.Kleisli;
import com.buildwithwaqar.patterns.functional.typeclass.OptionKind;
import dev.zazr.control.Option;
/** Problem: compose parseInt → *2 fallible pipeline. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Kleisli problem: fallible pipeline ===");
    Kleisli<OptionKind.Witness, String, Integer> parse = Kleisli.of(s -> {
      try { return OptionKind.lift(Option.some(Integer.parseInt(s))); }
      catch (Exception e) { return OptionKind.lift(Option.none()); }
    });
    Kleisli<OptionKind.Witness, Integer, Integer> twice = Kleisli.of(n -> OptionKind.lift(Option.some(n * 2)));
    var p = parse.andThen(OptionKind.MONAD, twice);
    System.out.println("ok=" + OptionKind.lower(p.apply("21")));
    System.out.println("bad=" + OptionKind.lower(p.apply("zz")));
  }
}
```

Walkthrough Output

```
=== Kleisli problem: fallible pipeline ===
ok=Some(42)
bad=None
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Kleisli.

Related Patterns

- Contrast Kleisli with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Kleisli → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.